

7-base-model-training

June 15, 2026

1 Base Model Training

We will train a basic model first without taking into account the spatial influence of the printhead.

1.1 MLP

1.1.1 Training

```
[ ]: import torch
import torch_geometric

print(torch.__version__)
print(torch_geometric.__version__)

[ ]: import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
import polars as pl

[ ]: PARQUET_PATH = "../data/interim/encoded-collapsed-waveform.parquet"

df = pl.read_parquet(PARQUET_PATH)

[ ]: feature_cols = [c for c in df.columns if not c.startswith("Value")]
value_cols = [c for c in df.columns if c.startswith("Value_")]

X = df[feature_cols].to_numpy()
y = df[value_cols].to_numpy()

print("df shape:", df.shape)
print("X shape:", X.shape)
print("y shape:", y.shape)
print("value_cols:", len(value_cols))
print(df[value_cols].shape)

[ ]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test  = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32)
y_test  = torch.tensor(y_test, dtype=torch.float32)

print(type(X_train), X_train.shape)
print(type(y_train), y_train.shape)

assert X_train.shape[0] == y_train.shape[0]
assert y_train.shape[1] == 1120

```

```

[ ]: class BaselineMLP(nn.Module):
    def __init__(self, input_dim, output_dim=1120):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, output_dim)
        )

    def forward(self, x):
        return self.net(x)

```

```

[ ]: model = BaselineMLP(X.shape[1], y.shape[1])

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

```

```

[ ]: epochs = 200

for epoch in range(epochs):
    model.train()

    preds = model(X_train)
    loss = criterion(preds, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 5 == 0:
        model.eval()
        with torch.no_grad():
            test_preds = model(X_test)

```

```

        test_loss = criterion(test_preds, y_test)

        print(f"Epoch {epoch} | Train Loss: {loss.item():.6f} | Test Loss: {
↪test_loss.item():.6f}")

torch.save({
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "loss": loss.item(),
}, "mlp_checkpoint1.pth")

```

From these results we can interpret that, since train and test are tracking closely, the model is not overfitting, validating the data preprocessing pipeline.

Following this, we want to check:

- Error per nozzle position (are some nozzles indices consistently worse?)
- Error per chip
- Error vs. dt2 (does the model struggle in high dt2 ranges?)

For this we will compute residuals and calculate the mean error per nozzle.

1.1.2 Validation

```

[ ]: import numpy as np

preds = model(X_test).detach().cpu().numpy()
y_true = y_test.detach().cpu().numpy()

residuals = preds - y_true

```

Overall error stats:

```

[ ]: print(f"Mean residual: {residuals.mean()}")
print(f"std: {residuals.std()}")
print(f"Mean Absolute Error: {np.abs(residuals).mean()}")

```

These results indicate that the model is not systematically wrong, however it is noisy.

Is the error structured across nozzles?

```

[ ]: nozzle_mean = residuals.mean(axis=0)

print(f" First 10 nozzles: {nozzle_mean[:10]}")
print(f"Last 10 nozzles: {nozzle_mean[-10:]}")
print(f"Nozzle mean std: {nozzle_mean.std()}")
print(f"Nozzle mean min: {nozzle_mean.min()}, Nozzle mean max: {nozzle_mean.
↪max()}")

```

The residuals show a mean standard deviation of 0.034 across nozzles, ranging from -0.16 to +0.11. This confirms spatial structure persists after MLP prediction. The model captured most global

trends but missed systematic spatial effects—placing it in the “in-between zone” where residuals are neither random nor dominated by strong missing physics. Spatial information could yield improvements.

Are some nozzles consistently bad?

```
[ ]: nozzle_abs = np.abs(residuals).mean(axis=0)

print(nozzle_abs[:10])
print(nozzle_abs.std())
print(nozzle_abs.max())
```

The maximum mean absolute error is ~0.193, with typical errors of 0.13–0.14. This indicates a detectable but not dramatic signal. The model shows some nozzle indices are harder to predict, maybe due to edge effects between chips, manufacturing asymmetry, or uncaptured dt2 interactions.

Nozzle variance score

```
[ ]: spatial_signal = np.std(residuals.mean(axis=0))
print(spatial_signal)
```

On the interpretation scale (<0.01 = no structure, 0.01 – 0.03 = weak, 0.03 – 0.06 = meaningful, >0.06 = strong), our result falls at the borderline of meaningful spatial structure. The residuals contain real spatial correlation.

1.1.3 Visual Representation

```
[ ]: import matplotlib.pyplot as plt

mean_nozzle_error = residuals.mean(axis=0)

plt.figure()
plt.plot(mean_nozzle_error)
plt.title("Mean residual per nozzle index")
plt.xlabel("Nozzle index (0-1119)")
plt.ylabel("Mean error")
plt.show()
```

```
[ ]: plt.figure()
plt.imshow(residuals[:200], aspect='auto')
plt.title("Residuals (first 200 samples)")
plt.xlabel("Nozzle index")
plt.ylabel("Sample index")
plt.colorbar()
plt.show()
```

1.1.4 Conclusion

Since structure is ordered, interactions are local, and chips are repeating blocks, we will attempt to use a Convolutional neural Network over the nozzle axis before jumping into spatial graphs or GNNs.

1.2 Convolutional Neural Network

For this experiment, we will predict full nozzle profile per experiment, while respecting chip structure. For each sample (30 chips, 1120 nozzles per chip, total = 33600 outputs) we will structure it as:

(30, 1120) per sample

and convolve along nozzle axis.

We treat each chip as an independent 1D signal first, because your experiment description says:

- chips share global {V, Fr}
- chips are physically adjacent
- but nozzle-level effects dominate locally

So we will do CNN inside chips first, then optional cross-chip mixing.

```
[ ]: value_cols = [f"Value_{i:03d}#_mean" for i in range(1120)]

missing = [c for c in value_cols if c not in df.columns]
print(len(missing), missing[:10])
```

```
[ ]: chip_size = 1120
     n_chips = 30

     y = df[value_cols].to_numpy()

     print(y.size)
     print(y.shape)
```

```
[ ]: y = y[:, :, None] # (N, 1120, 1)
```

```
[ ]: class CNN1D(nn.Module):
      def __init__(self, input_dim=38, output_dim=1120):
          super().__init__()

          self.fc = nn.Sequential(
              nn.Linear(input_dim, 128),
              nn.ReLU(),
              nn.Linear(128, 256),
              nn.ReLU()
          )

          self.expand = nn.Linear(256, output_dim)

          self.conv = nn.Sequential(
              nn.Conv1d(1, 8, kernel_size=5, padding=2),
              nn.ReLU(),
              nn.Conv1d(8, 1, kernel_size=5, padding=2)
```

```

    )

    def forward(self, x):
        x = self.fc(x)
        x = self.expand(x)
        x = x.unsqueeze(1)  # (N, 1, 1120)
        x = self.conv(x)
        return x.squeeze(1)

```

```

[ ]: X = torch.tensor(X, dtype=torch.float32)
     y = torch.tensor(y, dtype=torch.float32)

```

```

[ ]: model = CNN1D(input_dim=X.shape[1])

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(30):
    model.train()

    preds = model(X_train)
    loss = criterion(preds, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 5 == 0:
        torch.save({
            "epoch": epoch,
            "model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict(),
            "loss": loss.item(),
        }, "cnn_checkpoint.pth")
        print(epoch, loss.item())

```

```

[ ]: model.eval()
     with torch.no_grad():
         preds = model(X_test)
         val_loss = criterion(preds, y_test)

     print(val_loss.item())

```

1.3 Model Comparison: MLP vs CNN

The CNN achieved a test loss of 0.0412, while the MLP performed slightly better at approximately 0.0368. The key insight is that the CNN learned local smoothing patterns but failed to unlock

fundamentally new signal, generalizing slightly worse than the MLP.

Combining these results with earlier residual analyses confirms a consistent picture: the MLP already performs very well, the residual structure exists but remains weak, and the system behaves as mostly per-nozzle with global condition-driven physics.

Possible next steps:

- Improve the MLP through better regularization (dropout, weight decay), feature interactions, or residual connections
- Try a hybrid approach: MLP baseline with a small CNN learning residual corrections